# HTML . CSS

EINFÜHRUNG IN DIE WEBENTWICKLUNG

# Was haben wir vor?

| Montag | Dienstag | Mittwoch | Donnerstag | Freitag |
|---|---|---|---|---|
| Eröffnung;<br>Installationen;<br>Request-Response | Wiederholung<br>Wahl Hörsaal-sprecher<br> <p> | Wiederholung<br>Kind-, Geschwister-, Nachfahren-Selektoren | Besprechen des Praxisprojektes | Wiederholung<br>Verlinkungen |
| Elemente<br><p>, <h1>, <br>, <hr> | Box-Modell<br>margin: - padding: -<br>width: - height: | Listen<br>Navigationsleiste<br>Formulare | Tabellen | Nutzung von Frameworks |
| Zeichenforma-tierungen | Inline vs. Internal vs. External CSS | Praxisprojekt I:<br>apple-Seite | Bilder<br>Hintergründe<br>Farben | Responsives Webdesign<br>Media-Queries |
| Attribute<br><a>, <img><br>Eigenschaften<br>HTML-Grundgerüst | Selektoren &<br>Vererbung<br>Typ-, Klassen-, ID-Selektoren | Arbeiten mit dem Inspektor<br>Projekt planen<br>Projekt umsetzen | Praxisprojekt II: uma<br>position:<br>overflow:<br>background-image: | Zusammenfassung<br>Abschluss |

© Thomas Mahlau . Alle Rechte vorbehalten

# Installationen

ATOM . XAMPP

# Sublime Text & XAMPP

sublimetext.com

Installieren Sie SUBLIME TEXT, wenn dies noch nicht installiert ist.

Installieren Sie XAMPP (Apache, MySQL, PHP-Erweiterung etc.)

ACHTUNG:

Dateien, die mit Webserver angezeigt werden sollen, müssen im Verzeichnis

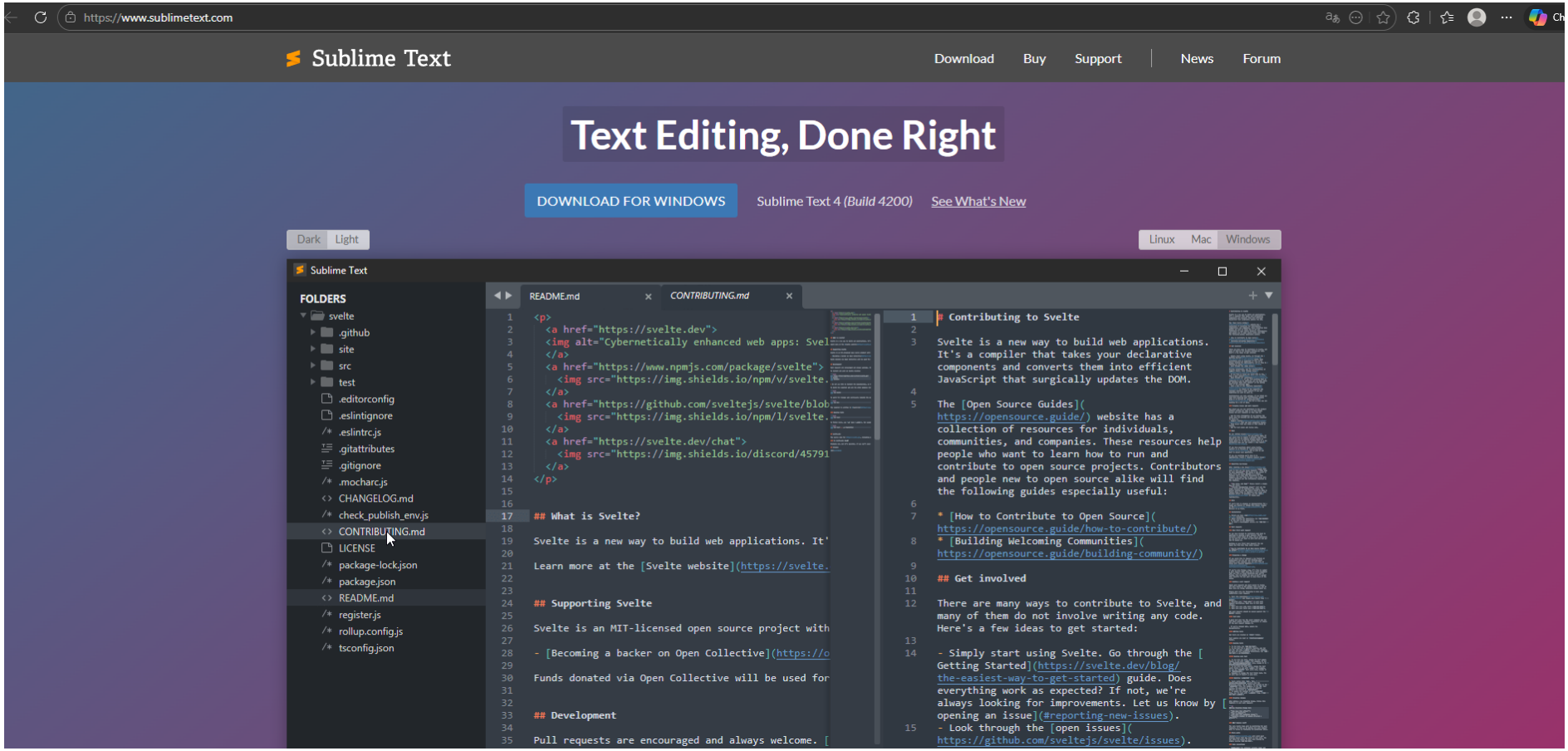
…\xampp\htdocs\<projektverzeichnis> abgespeichert sein.

**Erstellen Sie folgende Ordnerstruktur:**

```
<xampp>
    |--- <htdocs>
            |--- <htmlproject>
                    |--- <src>
                            |--- <css>
                            |--- <images>
```

# Sublime Text . Unser Editor

www.sublimetext.com



# XAMPP

https://www.apachefriends.org/download.html

https://www.apachefriends.org/download.html

Apache Friends | Download | Hosting | Community | About | Search.. | Search | EN

## Download

XAMPP is an easy to install Apache distribution containing MariaDB, PHP, and Perl. Just download and start the installer. It's that easy. Installers created using InstallBuilder.

### XAMPP for **Windows** 8.0.30, 8.1.25 & 8.2.12

| Version | | Checksum | | | Size |
|---|---|---|---|---|---|
| 8.0.30 / PHP 8.0.30 | What's Included? | md5 | sha1 | Download (64 bit) | 144 Mb |
| 8.1.25 / PHP 8.1.25 | What's Included? | md5 | sha1 | Download (64 bit) | 148 Mb |
| 8.2.12 / PHP 8.2.12 | What's Included? | md5 | sha1 | Download (64 bit) | 149 Mb |

Requirements More Downloads »

Windows XP or 2003 are not supported. You can download a compatible version of XAMPP for these platforms here.

### XAMPP for **Browsers** & 8.3.12

| Version | | Checksum | | | Size |
|---|---|---|---|---|---|
| 8.3.12 / PHP 8.3.12 | What's Included? | md5 | sha1 | Run in the browser | - |

### Documentation/FAQs

There is no real manual or handbook for XAMPP. We wrote the documentation in the form of FAQs. Have a burning question that's not answered here? Try the Forums or Stack Overflow.

- Linux FAQs
- Windows FAQs
- OS X FAQs

# XAMPP

XAMPP Control Panel v3.3.0 [ Compiled: Apr 6th 2021 ]

XAMPP Control Panel v3.3.0

Module

| Dienst | Modul | PID(s) | Port(s) | Aktionen | | | |
|---|---|---|---|---|---|---|---|
| ✕ | Apache | 42756<br>29564 | 80, 443 | Stoppen | Admin | Konfig | Logs |
| ✕ | MySQL | | | Starten | Admin | Konfig | Logs |
| ✕ | FileZilla | | | Starten | Admin | Konfig | Logs |
| | Mercury | | | Starten | Admin | Konfig | Logs |
| ✕ | Tomcat | | | Starten | Admin | Konfig | Logs |

Konfig
Netstat
Shell
Explorer
Dienste
Hilfe
Beenden

```
14:08:43 [main]    Initialisere Control Panel
14:08:43 [main]    Windows Version:  Enterprise  64-bit
14:08:43 [main]    XAMPP Version: 8.2.12
14:08:43 [main]    Control Panel Version: 3.3.0  [ Compiled: Apr 6th 2021 ]
14:08:43 [main]    Mit Administratorrechten gestartet - gut!
14:08:43 [main]    XAMPP Installationsverzeichnis: "c:\xampp\"
14:08:43 [main]    Voraussetzungen werden geprüft
14:08:44 [main]    Alle Voraussetzungen sind erfüllt
14:08:44 [main]    Initialisiere Module
14:08:44 [Tomcat]  Problem festgestellt!
14:08:44 [Tomcat]  Port 8080 in Benutzung von ""C:\Program Files\edb\pem\httpd\apache\bin\httpd.exe" -k runservice"!
14:08:44 [Tomcat]  Tomcat wird NICHT starten, wenn die konfigurierten Ports nicht frei sind!
14:08:45 [Tomcat]  Die blockierende Anwendung muss deinstalliert/deaktiviert/rekonfiguriert werden oder
14:08:45 [Tomcat]  Tomcat und das Control Panel müssen auf einen anderen Port zu lauschen
14:08:45 [main]    Starte Check-Timer
14:08:45 [main]    Control Panel bereit
14:08:47 [Apache]  Versuche Apache zu starten...
14:08:48 [Apache]  Statusänderung erkannt: gestartet
```

# Software zur Webentwicklung

1. Webserver

   (Apache, Internet Information Server, Apache Tomcat …)

2. Erweiterung des Webservers mit serverseitigen Programmiersprachen

   (PHP, Perl, Python, C# …)

3. Datenbankserver

   (MySQL/MariaDB, SQL-Server, PostgreSQL, Oracle …)

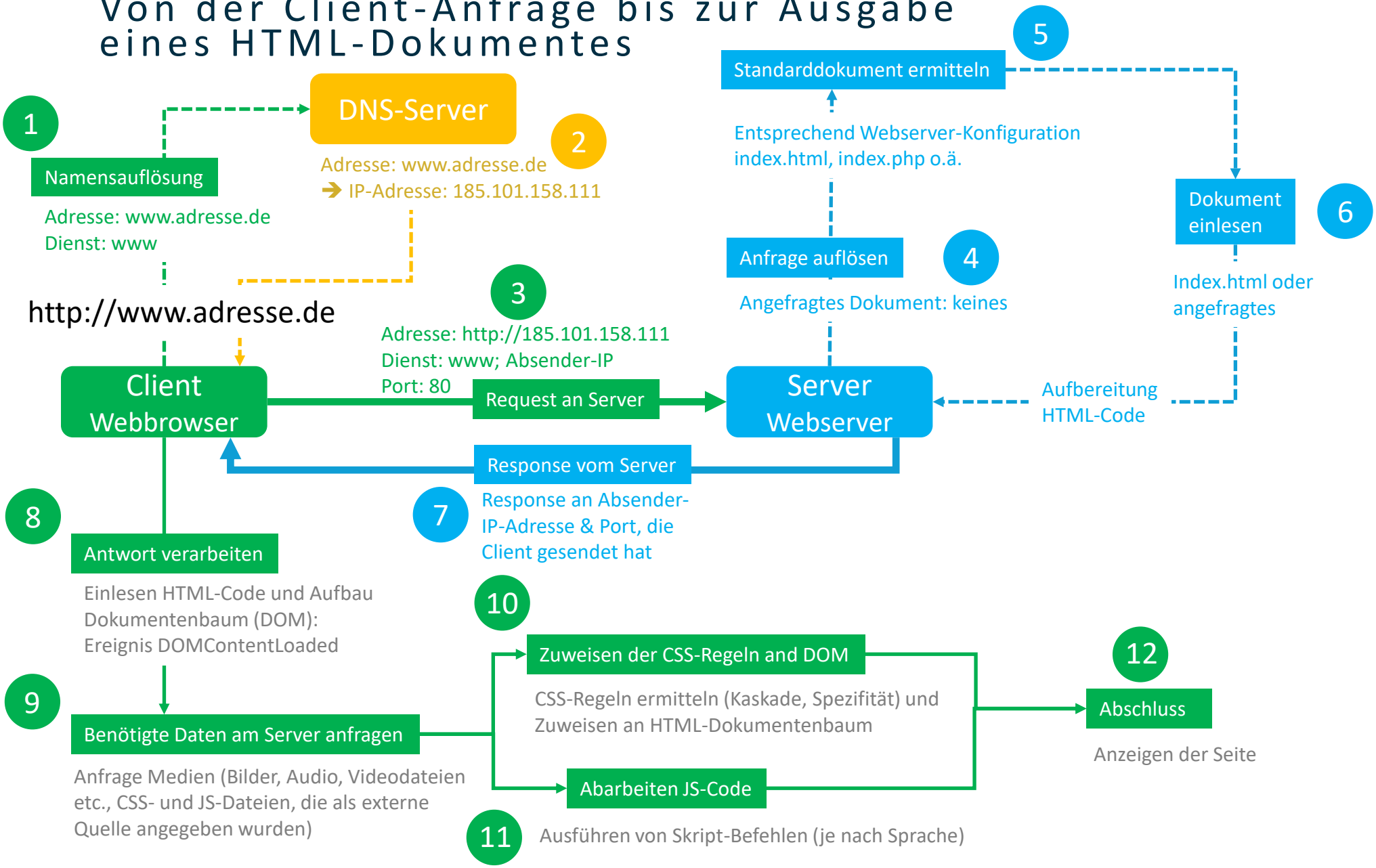
4. ggf. Mail- und FTP-Server …

   (Mercury (SMTP-Server), FileZilla (FTP) )

5. HTML (CSS-, JS-, PHP-) Editor

   (Atom, VS Code, PyCharm, Visual Studio, Sublime Text)

# Von der Client-Anfrage bis zur Ausgabe eines HTML-Dokumentes



# HTML Basic

Von Tags bis Grundstruktur

# Analogie

In Word gibt es den Unterschied zwischen:

- Zeichenformatierungen
- Absatzformatierungen
- Abschnittsformatierungen

Dieses Prinzip finden wir auch in HTML.

© Thomas Mahlau . Alle Rechte vorbehalten

# Analogie . Zeichenformatierung

## Dies ist eine Überschrift

**Dieser Text ist fett.**

*Dieser Text ist kursiv.*

Dieser Text ist <u>unterstrichen</u>.

Jedes Zeichen kann anders formatiert sein.

© Thomas Mahlau . Alle Rechte vorbehalten

# Analogie . Absatzformatierung

- Aufzählung 1
- Aufzählung 2

1. Aufzählung 1
2. Aufzählung 2

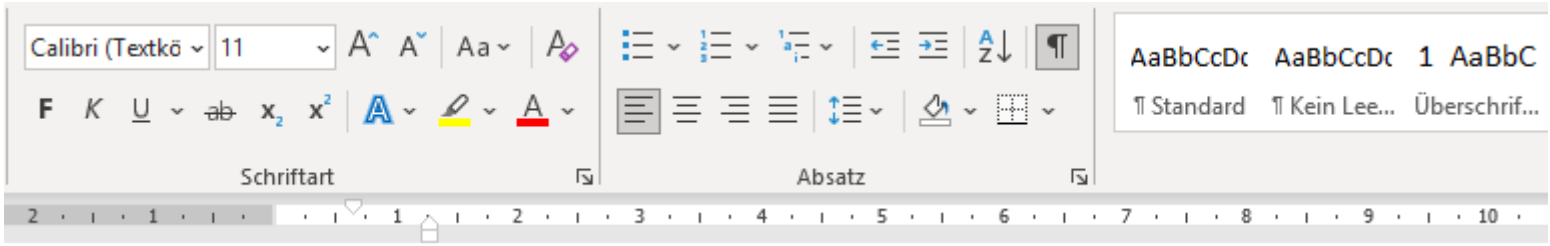
Dies ist ein Absatz, mit weiterem Text und einem linken Einsug. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Dieser Absatz hat einen Rahmen. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

© Thomas Mahlau . Alle Rechte vorbehalten

# Ende der Analogie

- HTML passiert online (erfordert Browser zur Interpretation des HTML, einen Webserver zur Bereitstellung in HTML, i.d.R. Datenbankserver für die Bereitstellung der zur Anzeige vorgesehenen Daten).
- HTML erstellen entspricht nicht der Arbeit mit einem TexterVERARBEITUNGSprogramm.
- Die Nutzerumgebungen (Browser) sind unterschiedlich.
- HTML wird i.d.R. nicht allein verwendet, sondern arbeitet zusammen mit APIs, Datenbanken, Frameworks …
- HTML ist „handmade“. Textverarbeitungen sind GUIs.




© Thomas Mahlau . Alle Rechte vorbehalten

# HTML . Einführung

HTML steht für:

- **Hyper Text Markup Language**

  ➔ Auszeichnungssprache vs. Skript- vs. Programmiersprache

- **Hyper Text**

  Verknüpfte Elemente (Links)

- **Markup Language**

  Auszeichnungssprache, in der beschrieben wird, wie eine Webseite aussehen soll

- Browser interpretiert HTML und zeigt dann die Webseite an

# HTML . Einführung

HTML ist genaugenommen:

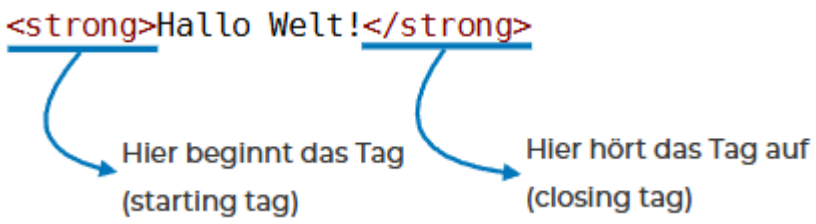
- Eine ganz normale *Textdatei*
- Dateiendung: `.html`
- Enthält spezielle Steuerbefehle & *Tags*
- *Steuerbefehle* werden vom Browser interpretiert
  - z.B. werden mehrere Leerzeichen hintereinander durch Browser auf ein Leerzeichen gebracht.
- Letztendlich wird die Seite dann im Browser angezeigt
- Moderne Browser können Html-Seite auch anzeigen, die direkt aus dem *Dateisystem* heraus geöffnet wurden.
- Im Echtbetrieb spielt der *Web-Server* eine große Rolle.
  Dieser übersetzt API-Code in HTML.

# Elemente

Die Tags

# HTML . Tag

Beispiel: Das -Tag zum *Fettschreiben* von Zeichen




Verschachtelung von Tags

Beispiel: -Tag (emphasis) zur *Kursivschreibung* zund -Tag

```
<em>Hallo</em> <strong>Welt</strong>!
```

```
<em>Hallo<strong>Welt</strong></em>!
```

# HTML . Tag

- HTML-Tags dürfen beliebig ineinander verschachtelt werden
- Jedes Tag muss geschlossen werden (bis auf die Ausnahmen)
- Alle KIND-Elemente müssen geschlossen werden BEVOR ELTERN-Elemente geschlossen werden.

Beispiele

Erlaubt

```
<strong><em>Hallo Welt!</em></strong>
```

Verboten

```
<strong><em>Hallo Welt</strong></em>
```

```
<em><strong>Hallo Welt!</em>
```

Browser versucht immer das Beste – auch aus Fehlern – zu machen.

Sollten Sie sich aber nicht drauf verlassen!

Aufgabe_01

# Textstrukturierung I

Überschriften . Paragraphen . Zeilenumbrüche . Horizontale Linien

# HTML . Überschriften

Insgesamt 6 Standard-Headline-Tags:

<h1>…</h1>

<h2>…</h2>

<h3>…</h3>

<h4>…</h4>

<h5>…</h5>

<h6>…</h6>

Überschriftenformatierung – Unterschiede liegen in den Schriftgrößen

Aufgabe_02

# HTML . Absätze . <p>-Tag

Lorem·ipsum·dolor·sit·amet,·consectetur·adipisicing·elit,·sed·do·eiusmod·tempor·incididunt·ut·labore·et·dolore·magna·aliqua.Ut·enim·ad·minim·veniam,·quis·nostrud·exercitation·ullamco·laboris·nisi·ut·aliquip·ex·ea·commodo·consequat.·Duis·aute·irure·dolor·in·reprehenderit·in·voluptate·velit·esse·cillum·dolore·eu·fugiat·nulla·pariatur.·Excepteur·sint·occaecat·cupidatat·non·proident,·sunt·in·culpa·qui·officia·deserunt·mollit·anim·id·est·laborum.¶

Lorem·ipsum·dolor·sit·amet,·consectetur·adipisicing·elit,·sed·do·eiusmod·tempor·incididunt·ut·labore·et·dolore·magna·aliqua.·Ut·enim·ad·minim·veniam,·quis·nostrud·exercitation·ullamco·laboris·nisi·ut·aliquip·ex·ea·commodo·consequat.·Duis·aute·irure·dolor·in·reprehenderit·in·voluptate·velit·esse·cillum·dolore·eu·fugiat·nulla·pariatur.·Excepteur·sint·occaecat·cupidatat·non·proident,·sunt·in·culpa·qui·officia·deserunt·mollit·anim·id·est·laborum.¶

Grundaufbau: `<p> Inhalte </p>`

- Bildet eine Texteinheit
- Erzwingt Zeilenumbruch – führt entsprechend der Grundeinstellungen auch zu Abständen zwischen Absätzen
- Gibt Möglichkeiten zur Gestaltung über STYLE = “….“

Beispiele:

```
<p style=“….“>Inhalt</p>
<p style=“color:red;“>Der angezeigte Text wird rot</p>
<p style=“font-size:150%;“>Der angezeigte Text erscheint 1,5mal so groß wie Standard</p>
<p style=“line-height:20px;“>Der Zeilenabstand beträgt 20px</p>
<p style=“line-height:20px;color:grey;font-family:Helvetica;“>Der Zeilenabstand
    beträgt 20px; Schriftfarbe: grau; Schriftart: Helvetica</p>
```

Es gibt zahlreiche weitere Formatierungsmöglichkeiten!

Aufgabe_03

# HTML . Zeilenumbruch

- HTML-Text wird – egal wie eingegeben – fortlaufend angezeigt.
- Mehrfache Leerzeichen hintereinander werden auf ein Leerzeichen verkürzt.
- Um ein Zeilenende zu erzeugen benutzen Sie <br>.

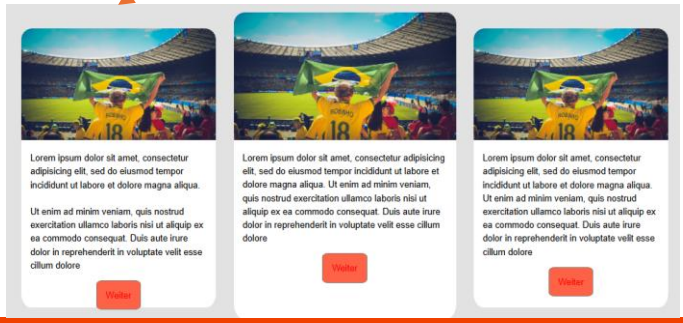
  Hiermit wird die Zeile abgeschlossen.
- Eine „Leerzeile“ erzeugen Sie durch zwei aufeinanderfolgende <br>-Tags
- M.a.W.: Nach einem <br> beginnt eine neue Zeile.
- Ein <br>-Tag erzeugt KEINEN Absatz!

Beispiel:

```
<p> Lorem ipsum dolor sit amet, consectetur adipisicing elit,
    sed do eiusmod tempor incididunt ut labore et dolore magna
    aliqua.<br><br>Ut enim ad minim veniam, quis nostrud exercitation
    ullamco laboris nisi ut aliquip ex ea commodo consequat.
    Duis aute irure dolor in reprehenderit in voluptate velit
    esse cillum dolore
</p>
```

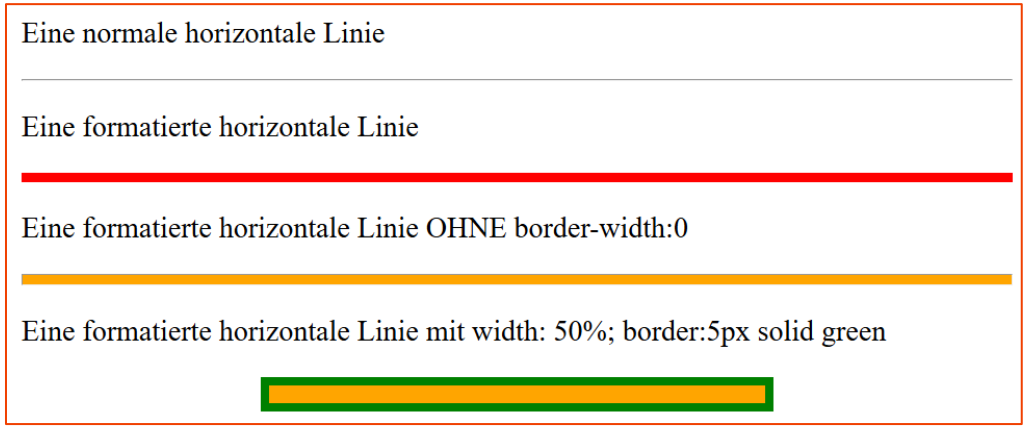
Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore



# HTML . Horizontale Linie

- Text kann außerdem grafisch durch horizontale Linien strukturiert werden.
- Hierzu benutzen Sie den <hr>-Tag
- Dieser ist ein sog. „Blockelement“ – nimmt also die gesamte Breite des Displays in Anspruch.
- Wie der <br>-Tag ist auch der <hr>-Tag ein selbstschließender Tag.
- Der <hr>-Tag kann formatiert werden.




Beispiel:

```
<p>Eine normale horizontale Linie</p>
<hr>
<p>Eine formatierte horizontale Linie</p>
<hr style="height:5px;border-width:0;background-color:red">

<p>Eine formatierte horizontale Linie OHNE border-
width:0</p>
<hr style="height:5px;background-color:orange">

<p>Eine formatierte horizontale Linie mit width: 50%;
border:5px solid green</p>
<hr style="height:10px;border: 5px solid green;
width:50%;background-color:orange">
```

# Textstrukturierung II

… durch Textformatierung

# HTML-Tag . Zeichenformatierung

- Text kann direkt durch Tags im Text formatiert werden.
- Es gibt hierzu immer einen öffnenden und schließenden Tag, der den Inhalt umfasst.

Beispiele:

```
Lorem <strong>ipsum dolor</strong> sit amet,
<u>consectetur adipisicing</u> elit,
sed do <b><i>eiusmod</i> tempor</b> incididunt <em>ut
<b>labore</b> <u>et <b>dolore</b> magna</u></em>
aliqua.
```

Lorem **ipsum dolor** sit amet, consectetur adipisicing elit, sed do ***eiusmod*** **tempor** incididunt *ut* ***labore*** *et* ***dolore*** *magna* aliqua.

textformatierung.html

# HTML . Schrift formatieren

| Tag | Bedeutung | Beispiel | Ausgabe |
|---|---|---|---|
|  | formatiert Text fett | Ich bin fett formatiert. | **Ich bin fett formatiert** |
|  | formiert ebenfalls fett | Ich bin fett formatiert. | **Ich bin fett formatiert** |
| <u> | unterstreicht Text. | <u>Ich bin unterstrichen.</u> | <u>Ich bin unterstrichen.</u> |
|  | Kursivschreibung | Ich bin kursiv. | Ich bin kursiv. |
|  | Kursivschreibung | Ich bin kursiv. | Ich bin kursiv. |
| <br> | Zeilenumbruch | Erste Zeile<br>Zweite Zeile | Erste Zeile<br>Zweite Zeile |
| <hr> | Horizontale Linie. Selbstschließend, kein Inhalt | <hr> | ---------- |
| <h1> | Überschrift erster Ordnung; <h1> bis <h6> | <h1>Überschrift 1</h1> | **Überschrift** |

Aufgabe_02-2

© Thomas Mahlau . Alle Rechte vorbehalten

# Attribute

Spezifikationen für das Element

# HTML-Tag . Attribute . Verlinkung

Das <a>-Tag dient der Verlinkung von Seiten miteinander.

Beispiel:

```
<a>google aufrufen</a>
```

Problem:
Keine Angabe, wohin verlinkt werden soll.

Lösung: Attribut, wohin verlinkt werden soll

```
<a href="https://www.google.com">google aufrufen</a>
```

Mehrere Attribute für ein Element sind erlaubt!

```
<a href="https://www.google.com" target="_blank">google aufrufen</a>
```

# HTML . Verlinkungen . <a>-Tag

Grundaufbau:

```
<a href=“NeueSeite.html“>Zur neuen Seite</a>
```

Webanwendungen verzweigen auf weitere oder Detailseiten mit Hilfe des <a>-Tags.

Wohin verlinkt wird, muss angegeben werden. Hierdurch wird die aktuelle Seite verlassen.

Es können unterschiedliche Objekte verlinkt werden:

- Text
- Bilder
- Grafische Objekte
- Symbole …

Aufgabe_05

# HTML . Bilder

*Pixabay.com liefert lizenzfreie Bilder – ACHTUNG: Lizenzbedingungen*

Grundaufbau:

```
<img src="./src/images/fussball.png" style="height:100px;"
alt="Fussballbild">
```

Hat keinen Schluss-Tag! (Da keinen Inhalt. „Selbstschließendes Tag.“)

Beachte: Angabe von *height* UND *width* führt ggf. zur Verzerrung des Originalbildes. Werden nur *height* ODER *width* vorgegeben, verändert sich die Größe relational – eine Verzerrung findet dann nicht statt.

Element | Attribut | Eigenschaft | Wert

# Eigenschaften

Von Farben, Hintergründen, Ausrichtungen und einiges mehr

# HTML-Tag . Eigenschaften

```
<img src="bild.jpg" width="500" height="600" alt="Bild mit Mann">
```

alt="…" ist zwingend erforderlich, weil

"Except where otherwise specified, the alt attribute must be specified

and its value must not be empty" (https://html.spec.whatwg.org/#alt)

```
<p style="background-color:tomato; color:white">Text mit Hintergrund und
weißer Schrift</p>
```

Achtung: Anführungszeichen fassen die gesamte Style-Definition ein.

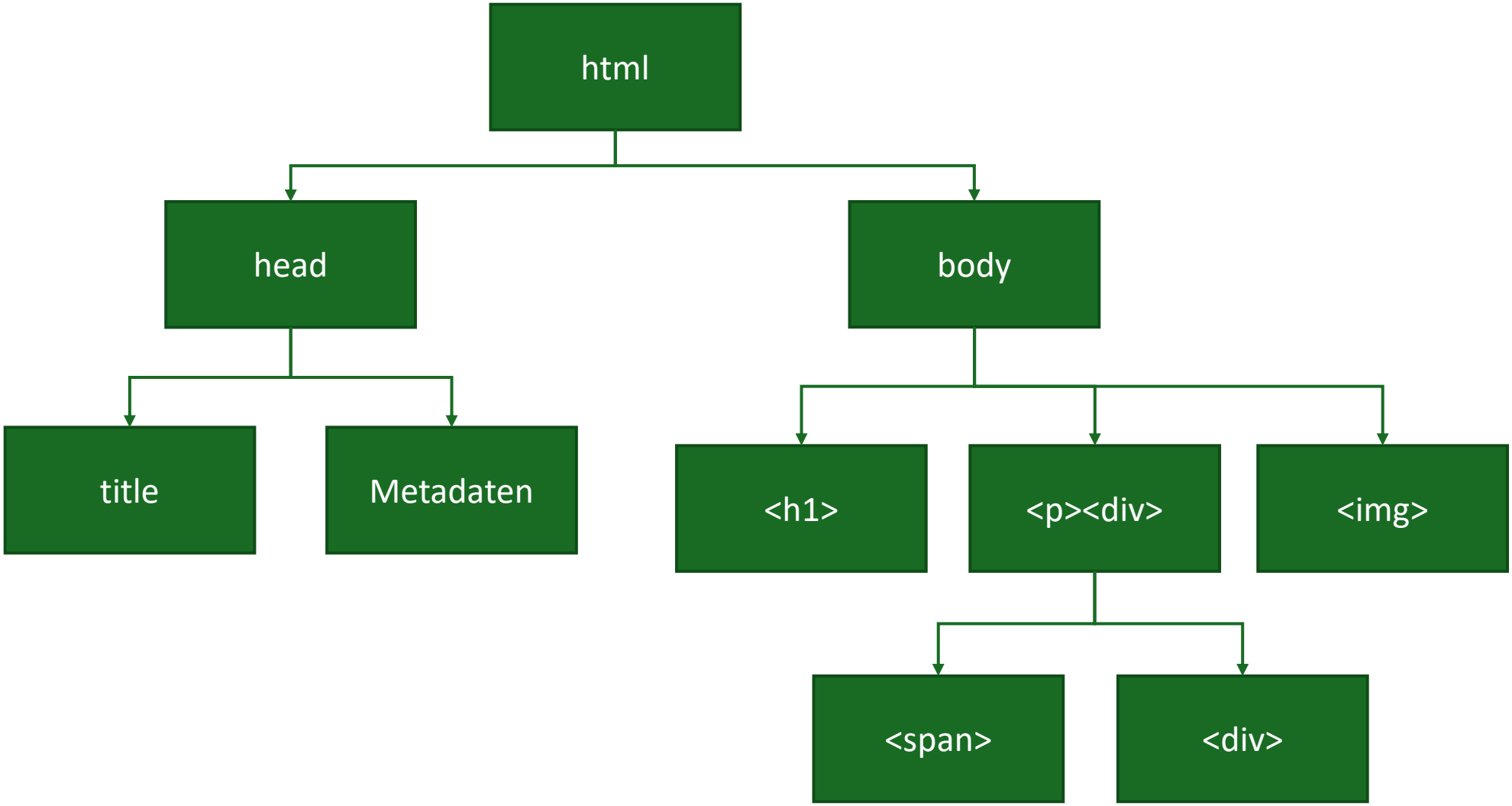
| Element | Attribut | Eigenschaft | Wert | |
|---|---|---|---|---|
| | | Eigenschaft | Wert | Inhalt |

Aufgabe_06

# HTML-Grundgerüst

Basics

# HTML . Grundgerüst



# HTML . Grundgerüst

```
<!DOCTYPE html>
<html lang="de">
        <head>
                <meta charset="utf-8">
                <link rel="stylesheet" type="text/css" href="styles.css">
                <title>
                        (Seitentitel, der im Tab des Browsers angezeigt wird)
                </title>
                <style>
                        (hier kommen die CSS-Definitionen rein – internal CSS)
                </style>
        </head>
        <body>
                (Hier kommt der Inhalt der Seiten rein – strukturiert durch <div>,<p> …)
        </body>
</html>
```

Aufgabe_07

# Strukturierung des Inhalts

Von Containern, Absätzen, Kindern, Geschwistern & anderen Familienangehörigen

# HTML . Von DIVs & Absätzen

```
<html>
    <head>
        (weitere sog. Meta-Tags /
Metadaten)
        <title>Mein Titel</title>
   </head>
   <body>
        <img src="bild.png">
        <h1>Seitenüberschrift</h1>
        <div>Erster Inhaltsblock</div>
        <div>
            <p>Absatz 1</p>
            <p>Absatz 2</p>
            <p>Absatz 3</p>
        </div>
        <div>Zweiter Inhaltsblock</div>
        <div>
            <div>Erstes Div im Div</div>
            <div>Zweites Div im Div</div>
        </div>
   </body>
```

DIVs dienen der zusätzlichen Strukturierung der Seite und erleichtern die Positionierung von Teilen der Seite enorm!

Für Standardelemente gibt es sog. „Semantische Tags“, die eigentlich auch DIVs sind. (dazu später)

➔ LOREM-Text

Aufgabe_07

# HTML . DIVs formatieren . Vererbung

```
<div style="background-color: red;"></div>

<div style="margin:2em;">Lorem …</div>

<div style="background-color: tomato; color: white; padding: 2em;">Lorem …</div>

<div style="background-color: green; width:200px; height: 200px;">Lorem …</div>

<div style="background-color: green; width:200px; height: 200px; margin: 2em;
    font-size=24px;">
   <p>Lorem …</p>
   <p>Lorem …</p>
   <p>Lorem …</p>
</div>

<div style="width: 200px; height: 200px; float:left;"><h1>Box 1</h1></div>
<div style="width: 200px; height: 200px; float:left;"><h1>Box 1</h1></div>
<div style="width: 200px; height: 200px; float:left;"><h1>Box 1</h1></div>

<p style="clear:both;"></p>

<p>weiter geht's</p>
```

Aufgabe_07-2

# Semantische Tags

<header>Logo, Name des Unternehmens, Banner</header>
<nav>Navigation der Seite</nav>
<main>Hauptteil</main>
<aside>Werbung o.ä.</aside>
<footer>Copyright, Navigation für Impressum, DSGVO, AGB …</footer>

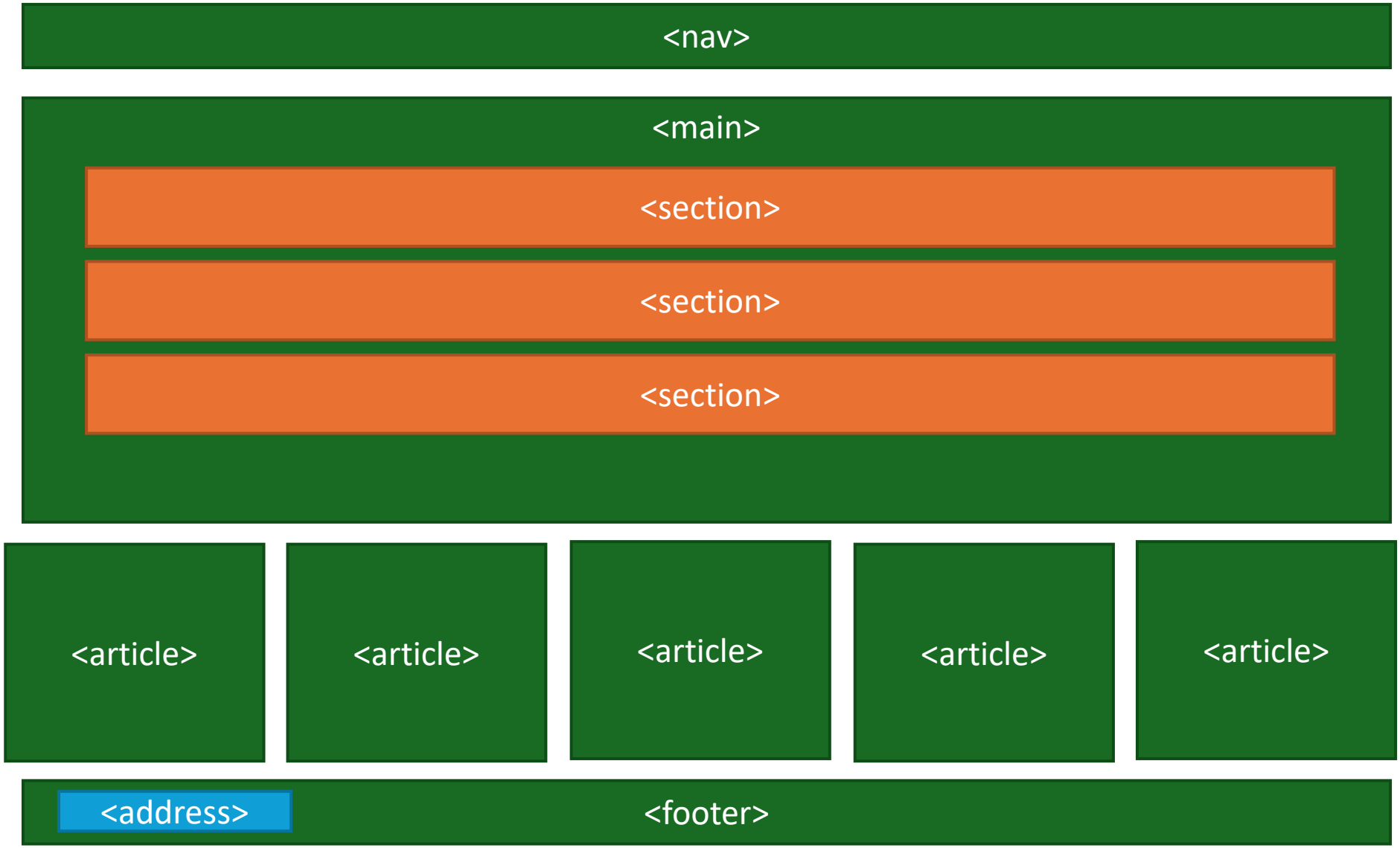
All diese Elemente haben keine optische Wirkung – dienen lediglich der Strukturierung.

<main> kann noch untergliedert sein durch:
<section>Container, der klarer ist als das allgemeine </section>
<article>Inhalt, der als in sich geschlossen angesehen wird: Forenbeiträge, Blogeinträge, Kommentare …</article>
<address>Kontaktinformationen zur Website oder zu Artikeln; häufig auch in Footer-Element</adress>

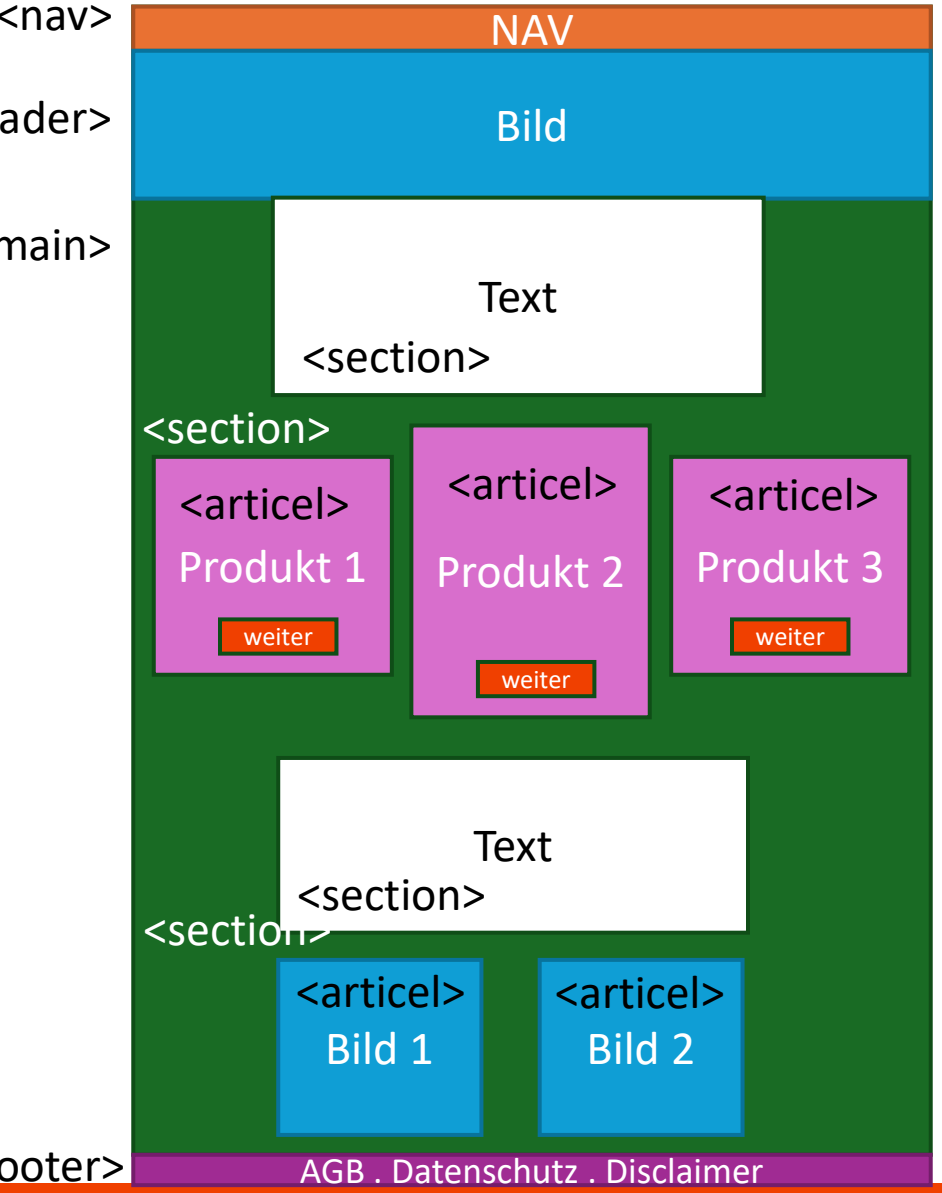
```
<header>…</nav>
<nav>…</nav>
<main>
<section>…</section>
<section>…</section>
<aside>…</aside>
<article>…</article>
<footer><address>…</adress></footer>
```

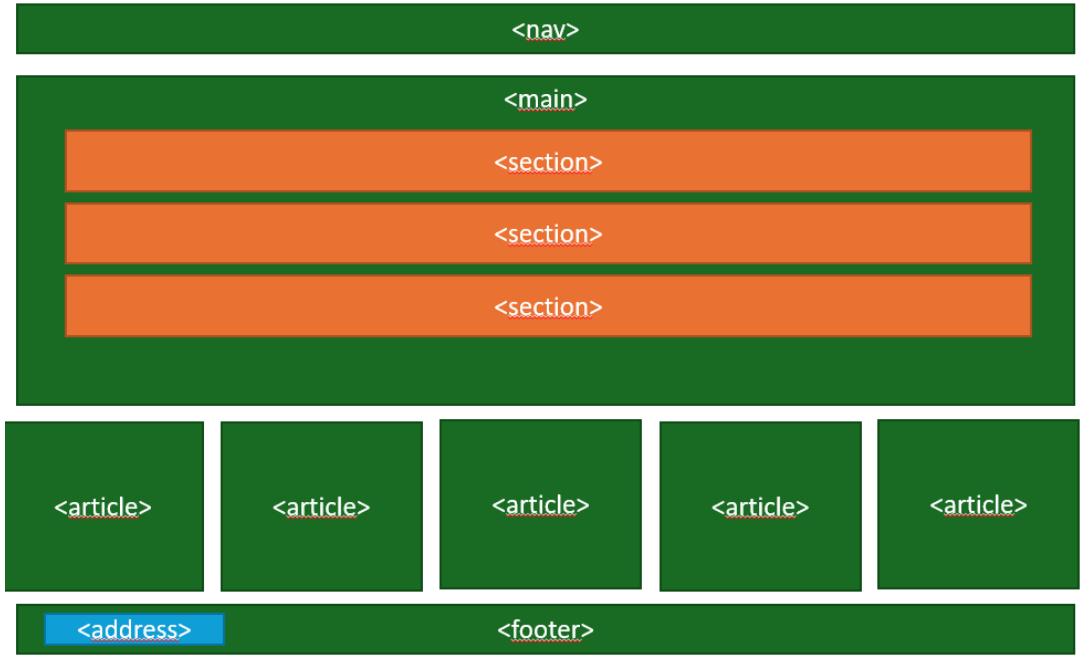
Aufgabe_17

# Semantische Tags … geben Struktur



# Übung . Semantische Tags



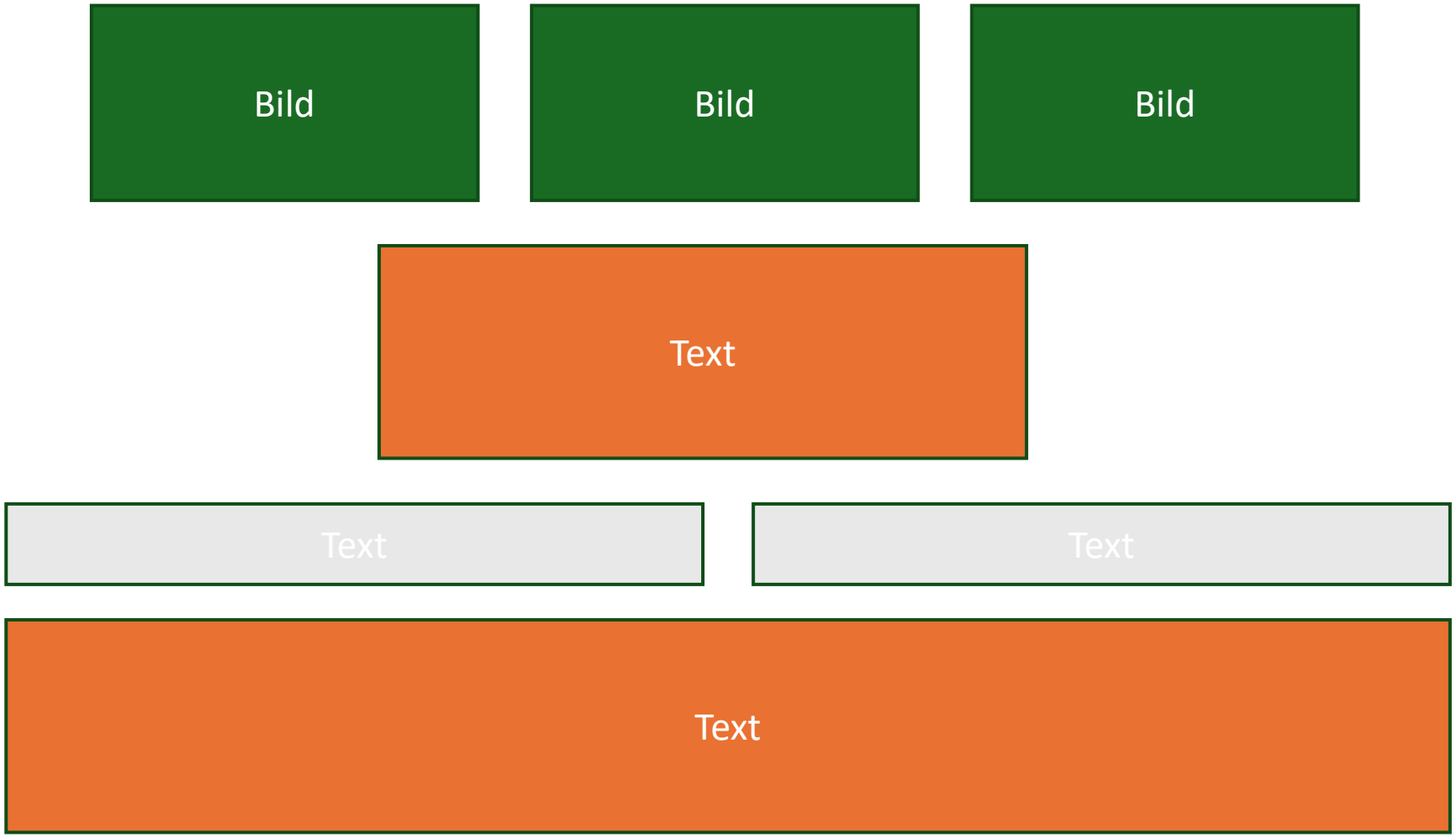




semantik.html

semantik.css

Aufgabe_17-2

# Übung . Semantische Tags



# DIV & Box-Modell

Immer untereinander ist langweilig.

# width: & height:

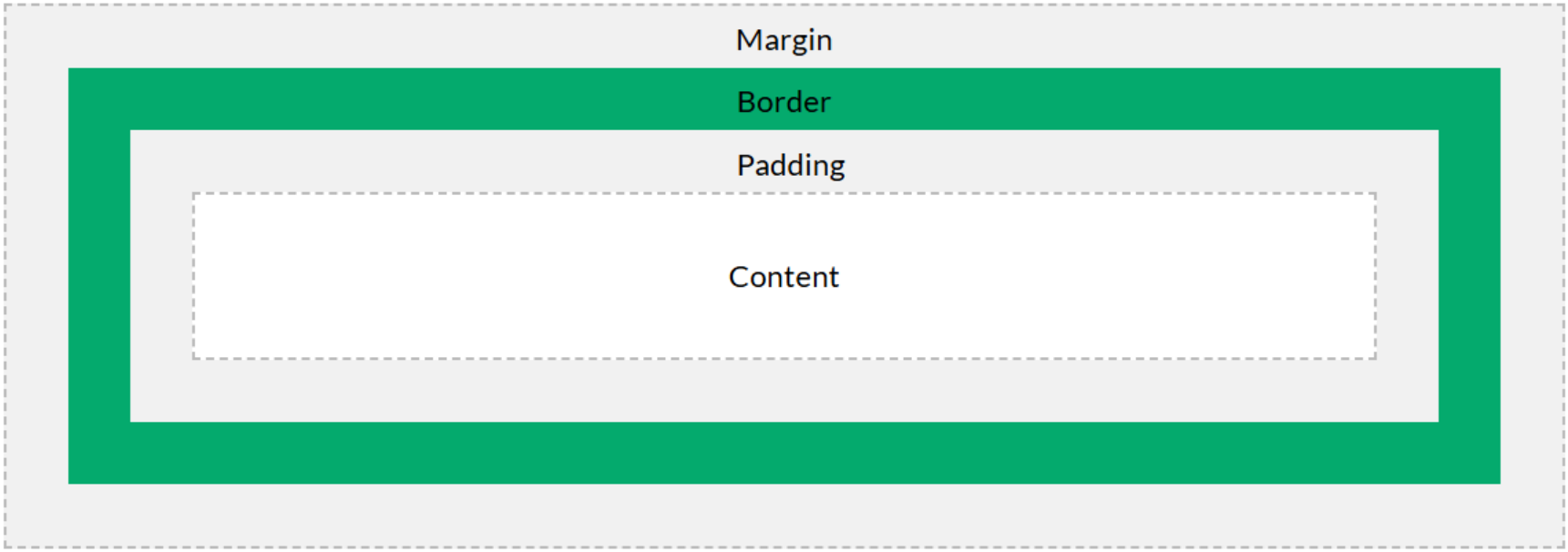
- Ein -Tag hat ohne Inhalt die Höhe 0 und die Breite 100%,
  da es ein sog. „Blockelement“ ist. M.a.W.: ohne Inhalt wird ein -Tag nicht dargestellt – auch wenn hierfür CSS-Definitionen vorliegen.
  Gleiches gilt übrigens auch für das <p>-Tag.
- -Tags können - auch ohne Inhalt – sichtbar gemacht werden, wenn die Höhe, Breite und z.B. Hintergrundfarbe festgelegt wird. Beispiel:

```
<div style=“width:300px; height:300px; background-color:tomato;“></div>
```

- Die Anordnung der aufeinander folgenden -Tags erfolgt standardmäßig *untereinander* – auch wenn die Breite reduziert wird.
  (Sie erinnern sich: Block-Element)

© Thomas Mahlau . Alle Rechte vorbehalten

# Box-Modell

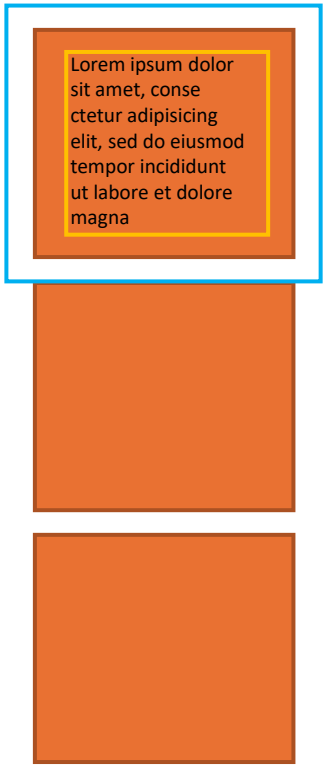


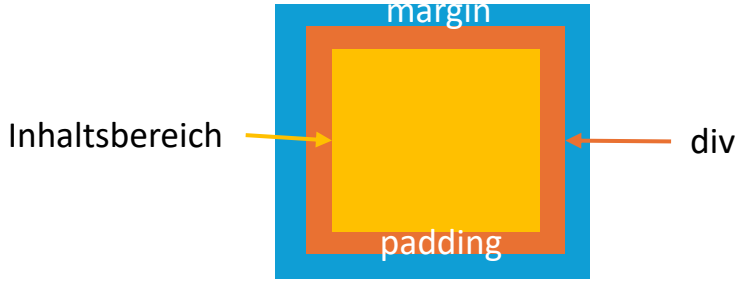

Das sog. Box-Modell beschreibt die regelmäßig relevanten Bestandteile in HTML-Elementen:

1. Content – Inhalt der Box mit Text oder Bildern
2. Padding – Der Abstand zwischen Content und Rand (innerer Abstand)
3. Border – Ein Rand, der um Padding & Content gezogen wird
4. Margin – Der Abstand außerhalb des Randes zu anderen Elementen (äußerer Abstand)

© Thomas Mahlau . Alle Rechte vorbehalten

# padding: & margin:




padding: und margin: können für alle 4 Seiten des Elements definiert werden:

padding: 1em; ➔ alle 4 Seiten erhalten den gleichen Wert

padding-top: 1em; ➔ nur Seite oben

padding-right: 1em; ➔ nur rechte Seite

padding-bottom: 1em; ➔ nur untere Seite

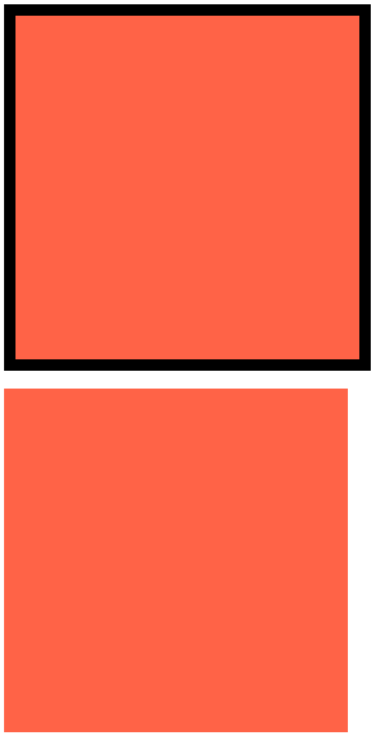
padding-left: 1em; ➔ nur linke Seite

padding: 1em 0.5em 1em 0.5em;

➔ 4-Seiten (im Uhrzeigersinn von oben) in einem Befehle

© Thomas Mahlau . Alle Rechte vorbehalten

# border:



Grundaufbau:

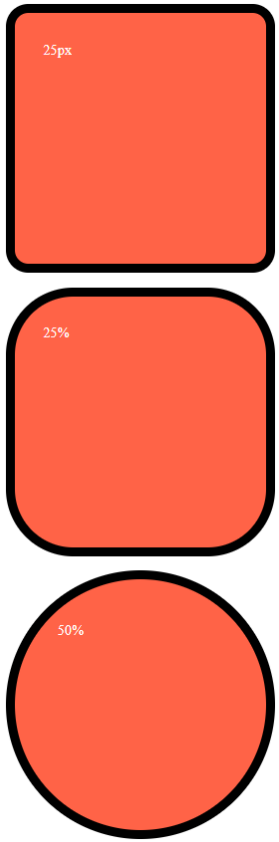
```
border: 10px solid black;
```

Alternative Border-Styles: dotted, dashed
Farben können mit Farbnamen, Hexwerten, rgb & rgba-Werten festgelegt werden.

WICHTIG:
Der Rahmen wird um das Element gezogen.
Die benötigte Fläche wird also größer!

© Thomas Mahlau . Alle Rechte vorbehalten

# border-radius:




Grundaufbau:

```
border-radius: 25px;
```

Werte können auch mit relativen Werten (%, em, rem) definiert werden.

Mit border-radius können Kreise erzeugt werden.

© Thomas Mahlau . Alle Rechte vorbehalten

# box-sizing: border-box

ohne box-sizing

mit box-sizing

Grundaufbau:
box-sizing: border-box;

I.d.R. wird dieses Einstellung am Anfang der CSS-Datei definiert:

```
* {
   box-sizing: border-box;
}
```

Mit dieser Grundeinstellung ersparen Sie sich viel Feintuning.

Der Rahmen wird jetzt nicht mehr um das Element gezogen. Die benötigte Fläche vergrößert sich nicht, sondern bleibt gleich.

© Thomas Mahlau . Alle Rechte vorbehalten

# box-shadow:

Syntax:

box-shadow: none|*h-offset v-offset blur spread color* |inset|initial|inherit;

```
h-offset:  erforderlich. Horizontaler (rechter) Abstand des Schattens. Negativer Wert stellt den
           Schatten auf der linken Seite dar.
v-offset:  erforderlich. Vertikaler (untere) Abstand des Schattens. Negativer Wert stellt den Schatten
           auf der oberen Seite dar.
blur:      optional. Radius der Unschärfe. Je höher der Wert, desto unschärfer.
spread:    Ausdehnung des Schattens.
color:     optional. Unterschiedliche Farbangaben sind erlaubt. (Safari fordert die Farbe!)
inset:     erzeugt einen inneren Schatten
inherit:   Einstellung des Elternelements werden geerbt.
```

Versuchen Sie mit unterschiedlichen DIV-Tags mit einer Backgroundfarbe, Höhe, Weite Ihrer Wahl folgende Einstellungen:

```
box-shadow: 10px 10px grey;
box-shadow: 50px 50px grey;
box-shadow: 20px 20px 10px grey;
box-shadow: 20px 20px 50px grey;
box-shadow: 20px 20px 10px 15px grey;
box-shadow: 20px 20px 20px 10px red;
box-shadow: 20px 20px 50px 10px pink inset;
box-shadow: none;
```

https://www.w3schools.com/cssref/playdemo.php?filename=playcss_box-shadow&preval=none

© Thomas Mahlau . Alle Rechte vorbehalten

# Absolute & relative Einheiten (Auswahl)

| Absolute Einheiten | | | Relative Einheiten | | |
|---|---|---|---|---|---|
| **Name** | **Einheit** | **Erklärung** | **Name** | **Einheit** | **Erklärung** |
| Pixel | px | Größe eines Pixels auf dem User-Monitor | Prozent | % | Relativ zum übergeordneten Element |
| Millimeter | mm | | Schriftgröße | em | Entspricht der vom User gewählten Schriftgröße, was in der Normaldarstellung 16px sind; relativ zum übergeordneten Element gerechnet. |
| Zentimeter | cm | | | rem | Wie em, jedoch relativ zum Root-Element (<html>) |
| Quarter-Millimeter | Q | Entspricht 1/40 eines Zentimeters | x-Höhe | ex | Entspricht der Höhe eines Kleinbuchstabens. |
| Zoll (inch) | in | Entspricht 2,54 cm | Viewport-Breite | vw | 1/100 der Breite des Viewports des User-Browsers |
| Punkt | pt | Entspricht 1/72 von einem Zoll (inch) | Viewport-Höhe | vh | 1/100 der Höhe des Viewports |
| Pica | pc | Entspricht 12 Punkten (12 pt) | Kleinere Seite des Viewports | vmin | 1/100 der kleineren Seite des Viewports |

Aufgabe_20

# CSS

Schluss mit Inline-Styling . Ab jetzt wird es übersichtlicher!

# Was ist CSS?

… Abkürzung für „Cascading Style Sheets“

- dt.: „Gestufte Gestaltungsbögen“ oder „kaskadierende Stilvorlagen“
- Mit CSS werden die Gestaltungsanweisungen für HTML erstellt
- CSS dient der Trennung von Inhalten (HTML) und Gestaltung (CSS)
- CSS kann auf 3 Arten festgelegt werden. (inline – internal – external)
- Grundsätzliches Ziel ist es, dass Inhalt (HTML) von Gestaltung (CSS) streng getrennt wird!
- Vorteil dieser Trennung:
  - leichtere Wartbarkeit der Seite,
  - geringere Fehleranfälligkeit,
  - strukturierte Programmierung und
  - Wiederverwertbarkeit einmal definierter Styles

# 3 Arten von CSS

## Inline vs. Internal vs. External Styles

CSS-Festlegungen können
a) direkt im Tag (s. oben) [INLINE]
b) innerhalb des `<style>`-Tags im `<head>`-Bereich [INTERNAL]
c) in separaten Dateien, die dann im `<head>`-Bereich eingebunden werden
definiert werden. [EXTERNAL]

Man unterscheidet folgende Definitionsvarianten für die Festlegungen im <style>-Tag:

1. Tag-Styles

```
p {
  color: red;
  font-size:15px;
  }
```

wirkt direkt auf jeden <p>-Tag, ohne weitere Angaben

Selektoren

2. Klassen

```
.meineKlasse {
  color: blueviolet;
  font-size:17px;
  }
```

Aufruf mit z.B.

```
<div class="meineKlasse">…</div>
```

3. ID

```
#meinStyle {
  color: darkgreen;
  font-size:20px;
  line-height: 1.4em;
  margin: 0 20% 0 20%;
  }
```

Aufruf mit z.B.

```
<div id="meinStyle">…</div>
```

# 3 Arten von CSS

## Inline vs. Internal vs. External Styles

HTML

```html
<p style="color:red;font-size:15px;">Absatz 1 - Schriftfarbe: rot; Schriftgröße: 15px</p>

<div class="meineKlasse">DIV mit Klasse formatiert - Schriftfarbe: rot; Schriftgröße: 17px</div>

<div id="meinStyle">Einmalige Formatierung mit ID-Style</div>

</body>
</html>
```

CSS

```css
.meineKlasse {
    color: blueviolet;
    font-size: 17px;
}

#meinStyle {
    color: darkgreen;
    font-size: 20px;
    line-height: 1.4em;
    margin: 0 20% 0 20%;
}
```

Anzeige in Browser

Absatz 1 - Schriftfarbe: rot; Schriftgröße: 15px

DIV mit Klasse formatiert - Schriftfarbe: rot; Schriftgröße: 17px

Einmalige Formatierung mit ID-Style

# Inline CSS

- Inline CSS funktioniert über das style-Attribut
- Grundsätzlich kann jedem HTML-Element das style-Attribut zugewiesen werden. Wie gesagt: Das sollten Sie aber NICHT tun.

Aufgabe: Weisen Sie die unten stehenden Begriffe den Bestandteilen der nachfolgenden Anweisung zu:

```
<p style="color: blue;">Inhalt</p>
```

Element | Attribut | Eigenschaft | Wert | Inhalt

# Internal CSS: Style-Bereich

```
<!DOCTYPE html>
<!-- <html lang="de"> -->
  <head>
    <!-- <meta charset="utf-8"> -->
    <title>Intro</title>
    <style>

    </style>
  </head>
  <body>
    <h1>Einführung</h1>

    <em>Hallo</em> <strong>Welt</strong>

    <ol>
      <li>eintrag 1</li>
      <li>eintrag 1</li>
      <li>eintrag 1</li>
    </ol>
    <b>fette Schrift</b><br>
    <strong>auch fette Schrift</b>
  </body>
</html>
```

Style-Bereich

Beispiel:

```
<style>
  body {
    font-family: Helvetica;
    color: grey;
    font-size: 24px;
  }

  div {
    width: 70%;
    margin: auto;
    text-align: justify;
  }
</style>
```

Die Styles wirken sich auf alle Inhalte des <body> aus und wirken auf alle -Elemente.

# Internal CSS: (Typ-) Selektoren

```html
<!DOCTYPE html>
<!-- <html lang="de"> -->
  <head>
    <!-- <meta charset="utf-8"> -->
    <title>Intro</title>
    <style>
      h1 {
        color: blue;
      }
    </style>
  </head>
  <body>
    <h1>Einführung</h1>

    <em>Hallo</em> <strong>Welt</strong>

    <ol>
      <li>eintrag 1</li>
      <li>eintrag 1</li>
      <li>eintrag 1</li>
    </ol>
    <b>fette Schrift</b><br>
    <strong>auch fette Schrift</b>
  </body>
</html>
```

Eigenschaften und Werte, die im Style-Bereich definiert werden, müssen mit Selektoren referenziert werden.

h1 als Selektor für alle h1-Elemente

Alle Überschriften, die mit h1-Tag umfasst sind, erhalten blaue Schriftfarbe.
Der HTML-Code bleibt „clean“ und Änderungen wirken auf ALLE h1-Tags.

Syntax von Internal & External CSS:

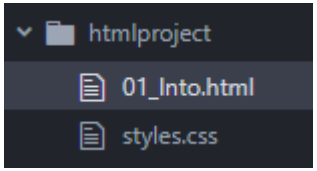
h1 { color: blue; }

WICHTIG: { ..: ...;}

{ }

Selektor Eigenschaft Wert : ;

# External CSS




Separate Dateien

```html
<!DOCTYPE html>
<!-- <html lang="de"> -->
  <head>
    <!-- <meta charset="utf-8"> -->
    <title>Intro</title>
    <link rel="stylesheet" type="text/css" href="styles.css">
  </head>
  <body>
    <h1>Einführung</h1>

    <em>Hallo</em> <strong>Welt</strong>

    <ol>
      <li>eintrag 1</li>
      <li>eintrag 1</li>
      <li>eintrag 1</li>
    </ol>
    <b>fette Schrift</b><br>
    <strong>auch fette Schrift</b>
  </body>
</html>
```

index.html

```css
h1 {
  color: blue;
  font-size: 20px;
}
```

styles.css

I.d.R. gibt es eine CSS-Datei, in der alle Definitionen erfolgen.
Wichtig ist die Referenzierung auf die CSS-Datei im <head>-Tag der HTML-Datei / Inhaltsdatei.

# External CSS

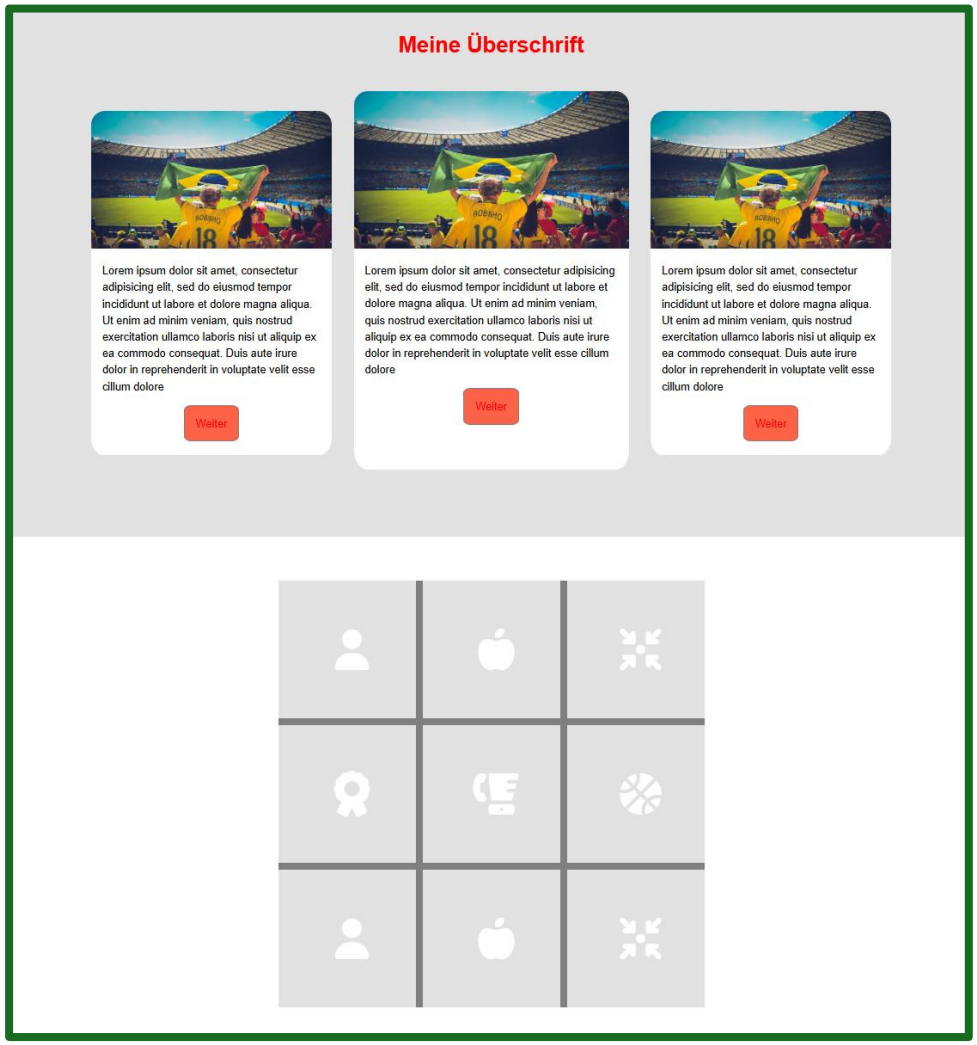
```
<head>
    <title>Meine Page</title>
    <link rel="stylesheet" type="text/css" href="styles.css">
    …
</head>
```

- <link>-Tag legt fest, wie das aktuelle Dokument (index.html) zu anderen in Beziehung steht.
- rel-Attribut bestimmt die Beziehung (Relation) zu der Datei, die eingebunden wird (hier: stylesheet)
- type-Attribut bestimmt die Art der Datei, die eingebunden werden soll (Text- bzw. CSS-Datei)
- href-Attribut gibt die URL zu einem anderen Dokument an

Aufgabe_15 bis 18

# Boxen . Boxen . Boxen

Gestalten Sie folgende Seite:




```
<script src="https://kit.fontawesome.com/69f45dc3fc.js" crossorigin="anonymous"></script>
```

1. Analysieren Sie den Aufbau der Seite
2. Legen Sie den semantischen Aufbau fest.
3. Vergegenwärtigen Sie sich die notwendigen Techniken.
4. Notieren Sie Fragen zur Realisierung.
5. Gestalten Sie die Seite mit externen Styles.

```
<i class="fa-sharp fa-solid fa-user iconclass"></i>
<i class="fa-solid fa-apple-whole iconclass"></i>
<i class="fa-solid fa-arrows-to-circle iconclass"></i>

<i class="fa-solid fa-award iconclass"></i>
<i class="fa-solid fa-blender-phone iconclass"></i>
<i class="fa-solid fa-basketball iconclass"></i>

<i class="fa-sharp fa-solid fa-user iconclass"></i></div>
<i class="fa-solid fa-apple-whole iconclass"></i>
<i class="fa-solid fa-arrows-to-circle iconclass"></i>
```

boxen.html

# TYPSELEKTOREN & VERERBUNG

ERBEN ODER NICHT ERBEN – DAS IST HIER DIE FRAGE

# HTML . Kleines Beispiel

Sie haben folgende CSS-Definitionen:

```
body {
   width: 80%;
   margin: auto;
   font-family: Helvetica;
   color: grey;
}

div {
   color: blue;
}
```

Im <body> steht folgender Code:

1 Die ist ein Absatz.

2 Hier kommt mein erstes Div

3
```
<div>
   <p>Ein Absatz im Div</p>
   <p>Noch ein Absatz im Div</p>
</div>
```

Welche Schriftfarbe, Breite, Ausrichtung haben die Einheiten 1 bis 3?

# HTML . Vererbung

HTML-Code ist immer hierarchisch aufgebaut:

```
<html>
   <body>
     <div> / <p> /  <img …> …
     <div>
       <p>…</p>
     </div>
    …
   </body>
</html>
```

Document Object Modell – DOM:

„Das DOM stellt das gesamte Webdokument als eine **hierarchische Baumstruktur** dar, in der jedes Element des Dokuments einem Knoten im Baum entspricht. Diese Struktur macht es möglich, **mit einzelnen Teilen eines Webdokuments** gezielt zu interagieren und diese zu manipulieren, was für die Erstellung interaktiver und dynamischer Webseiten unerlässlich ist.“
Quelle: https://www.impulsphase.de/wiki/dom

Daraus folgt:

- Alle Elemente erben von <html>.
- Alle Inhalte des <body> erben von den <body>-CSS-Festlegen (ggf. auch von <html>).
- <p> erbt von -Definitionen, wenn diese im  enthalten sind – sonst allein von <body> bzw. <html>.
- Werden für nachgeordnete Elemente besondere Festlegungen gemacht, überschreiben diese die übergeordneten.

# CSS-Selektoren – Teil I

Typ-Selektoren, Klassen-Selektoren, ID-Selektoren

# CSS . Typ-Selektoren

Wir haben schon mit Typ-Selektoren in unseren CSS-Festlegungen gearbeitet, ohne diese so zu nennen.

Beispiel:

```
div {
  width: 80%;
  margin: auto;
}
```

div ist der Typ-Selektor, der die CSS-Festlegungen für alle -Tags enthält.
Alle Elemente ( , <p>, , <h1>, <img>, <a> …) können über Typ-Selektoren im CSS gezielt gestaltet werden.
Festlegungen über Typ-Selektoren wirken sehr grundlegend, können aber jederzeit durch andere (detailliertere) Festlegungen überschrieben werden.

Möglich ist auch die Festlegung von Definitionen für mehrere Typ-Selektoren:

```
div, p, h1 {
  width: 80%;
  margin: auto;
}
```

Andere Tags (z.B.: <img>) sind dann nicht davon betroffen, sofern diese nicht von den o.g. „erben“.

# Formatierungen

w3schools.com

Eine hervorragende Sammlung von Hinweisen, Beispielen, Quizes etc. rund um alle relevanten Themen, die mit der Webentwicklung zu tun haben.

Hier finden Sie unzählige Formatierungsnuancen mit Aufstellungen der Varianten und leicht verständlichen Beispielen.

Die wichtigsten auf einen Blick:

```
margin
padding
color
background-color
border
height
width
font…(-style, -size, -family, -align)
```

Weitere und genaue Erklärungen hierzu finden Sie auf w3schools!

Aufgabe_09

# Listen

Geordnet, ungeordnet, als Navigationsleiste

# HTML . Listen

GEORDNETE LISTE.

Grundaufbau:

```
<ol>
    <li>Eintrag 1</li>
    <li>Eintrag 2</li>
    <li>Eintrag 3</li>
</ol>
```

<ol type="1">

ermöglicht die Änderung der Aufzählungs-zeichen.
Varianten:

- type="1"
- type="A"
- type="a"
- type="I"
- type="i"

Zur Erinnerung.

UNGEORDNETE LISTE.

Grundaufbau:

```
<ul>
    <li>Eintrag 1</li>
    <li>Eintrag 2</li>
    <li>Eintrag 3</li>
</ul>
```

<ul style="list-style-type:circle;">

ermöglicht die Änderung der Aufzählungs-zeichen.
Varianten:

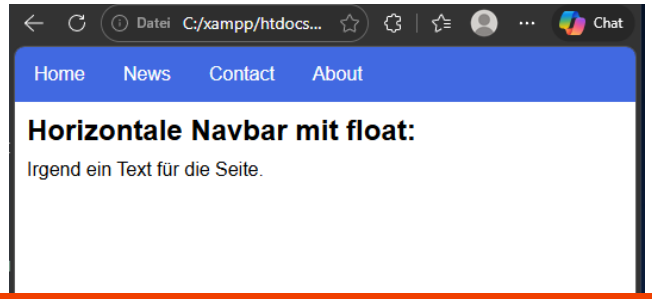
- circle
- square
- none

Aufgabe_08

# HTML . Listen für Navigationen

```html
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" href="navbar.css">
</head>
<body>

<ul>
  <li><a href="#home">Home</a></li>
  <li><a href="#news">News</a></li>
  <li><a href="#contact">Contact</a></li>
  <li><a href="#about">About</a></li>
</ul>

<h2>Horizontale Navbar mit float:</h2>

<p>Irgend ein Text für die Seite.</p>

</body>
</html>
```

```css

body {
  margin: 0;
  padding: 0;
  font-family: Helvetica;
}

h2, p {
  margin: 10px;
}

ul {
  list-style-type: none;
  margin: 0;
  padding: 0;
  overflow: hidden;
  background-color: royalblue;
}

ul li {
  float: left;
}

ul li a {
  display: block;
  color: white;
  text-align: center;
  padding: 14px 16px;
  text-decoration: none;
}

ul li a:hover {
  background-color: indianred;
}
```



# Formatierungen

w3schools.com

margin
Padding

```
margin: auto;
```

```
float: left;
 …
clear: both;
```

```
display: flex;
```

Aufgabe_09

# Symbole

https://www.mediaevent.de/tutorial/sonderzeichen.html

Diese Seite liefert eine sehr nützliche Aufstellung von Symbole, die auf unterschiedliche Weise aufgerufen werden können.

Beispiel: Anzeige des Copyrightzeichens durch
©
ODER
©

führen beide zur Darstellung des Symbols ©

Eine umfangreiche Icon-Sammlung liefert fontawesome.
Recherchieren Sie, welche Möglichkeiten des Einbindens Sie von fontawesome haben und nutzen sollten. Erstellen Sie eine kurze Übersicht auf einer PowerPoint-Folie.

Weitere Hinweise zur Nutzung der sog. Icon-Kits finden Sie unter:
https://docs.fontawesome.com/web/setup/use-kit/

Aufgabe_10

# FORMULARE

Jetzt wird es interaktiv !???

# <form> . <input> . GET . POST

```
<form action="" method="GET oder POST">
        <input type = "...">
</form>
```

© Thomas Mahlau . Alle Rechte vorbehalten

# Formulare . <input …>-Tag

Grundaufbau:

```
<form action="zielseite.php" method="POST"> (oder "GET")
        <input type="text" name="fname" placeholder="Ihr Nachname">
        <input type="email" name="femail" placeholder="Ihre Mailadresse">
            (überprüft Eingabeformat der Email und liefert Meldung bei
Fehlern bei Absenden)
        <input type="password" name="fpwd" placeholder="Ihr Passwort">
            (verhindert Klartextanzeige im Feld)
        <input type="submit" name="bsave" value="Speichern">
            (Submit-Button sendet Formularinhalte an Zielseite zur weiteren
                Verarbeitung)
</form>
```

ACHTUNG: Mit der GET-Methode oder ohne Angabe einer Methode (entspricht GET-Methodeneinstellung) werden die Feldinhalte in der Adresszeile des Browsers angezeigt.

Zur Verarbeitung vertraulicher Daten IMMER die POST-Methode verwenden.

Aufgabe_11

# Formulare . Radiobuttons

Grundaufbau:

```
<input type=“radio“ name=“fradio“ value=“Wert1“>
```

Beispiel:

```
<input type="radio" id="html" name="fav_language" value="HTML">
<label for="html">HTML</label><br>
<input type="radio" id="css" name="fav_language" value="CSS">
<label for="css">CSS</label><br>
<input type="radio" id="javascript" name="fav_language" value="Java
Script"><br>
<label for="javascript">JavaScript</label>
```

ACHTUNG: Namen aller Radio-Auswahlpunkte müssen gleich sein!
Unterschiede ergeben sich aus unterschiedlichen value-Angaben.

Auch dieser Input-Type muss innerhalb eines FORM-Tags aufgeführt werden, damit der gewählte Wert versandt werden kann.

Aufgabe_12

# Formulare . Checkboxen

```
<input type=“checkbox“ name=“meineCheck1“ value=“MeinWert1“>
<input type=“checkbox“ name=“meineCheck2“ value=“MeinWert2“>
<input type=“checkbox“ name=“meineCheck3“ value=“MeinWert3“>
```

Jedes Formularfeld muss einzeln ausgewertet werden, um den spezifischen Wert vermitteln zu können.

Beispiel:

```
<input type="checkbox" id="vehicle1" name="vehicle1" value="Bike">
<label for="vehicle1"> I have a bike</label><br>
<input type="checkbox" id="vehicle2" name="vehicle2" value="Car">
<label for="vehicle2"> I have a car</label><br>
<input type="checkbox" id="vehicle3" name="vehicle3" value="Boat">
<label for="vehicle3"> I have a boat</label><br>
```

ACHTUNG: Namen müssen unterschiedlich sein. Mehrfachauswahl ist so möglich.

Aufgabe_13

# Input-Types

- <input type="button">
- <input type="checkbox">
- <input type="color">
- <input type="date">
- <input type="datetime-local">
- <input type="email">
- <input type="file">
- <input type="hidden">
- <input type="image">
- <input type="month">
- <input type="number">
- <input type="password">
- <input type="radio">
- <input type="range">
- <input type="reset">
- <input type="search">
- <input type="submit">
- <input type="tel">
- <input type="text">
- <input type="time">
- <input type="url">
- <input type="week">

# Formularfeld-Attribute (Auswahl)

- required

```
<input type=“text“ name=“name“ required>
```

- placeholder

```
<input type=“text“ name=“vorname“ placeholder=“Vorname“>
```

- readonly

```
<input type="text" name="readonly" value="Das kann nicht geändert werden"
size="100px" readonly><br>
```

- disabled

```
<input type=“text“ name=“absender“  value=“noreplay@meinefirma.de“ disabled>
```

- autofocus

```
<input type=“text“ name=“erstesfeld“ autofocus>
```

- autocomplete

```
<input type="email" name="email" autocomplete>
```

# Block- & Inline-Elemente

## Block-Elemente:

Erzeugen eine Rechteck über die gesamte Displaybreite.

## Inline-Elemente:

Elemente, die auf einer Zeile eines Blockelements platziert werden können.
Text ist KEIN Inline-Element, da es überhaupt kein Element ist, sondern ein INHALT.

Diese Unterscheidung ist extrem wichtig, da diese die Anordnung des Inhaltes bzw. der Elemente maßgeblich beeinflusst.

# Block-Elemente & Inline-Elemente

## Block-Elemente

- h1, h2, h3, … h6
- p
- div
- ul, ol, dl
- li, dt, dd
- table, tr
- td

## Inline-Elemente

- img
- a
- br
- span
- b
- strong
- i
- em

## display-Eigenschaften:

- **display: block;**
  Elemente ist Blockelement (geht über gesamte Bildschirmbreite)
- **display: inline;**
  Elemente ist Inlineelement (keine Inhaltsunter-brechung)
- **display: inline-block;**
  alle Block-Elemente sind erlaubt; Elemente werden aber nebeneinander angeordnet.
- **display: flex;**
  alle Elemente werden nebeneinander angeordnet. Flex-Box wird angelegt.
- **display: grid;**
  alle Elemente werden nebeneinander angeordnet. Grid-Element wird angelegt.

Aufgabe_16

# Block-Elemente & der „Überlauf“

Beim Arbeiten mit Blockelementen mit definierter Höhe kommt es oft zu ungewollten Darstellungen.
Beispiel: Text „läuft über“.

Die Lösung liegt in der Eigenschaft `overflow`, die sowohl vertikal wie auch horizontal definiert werden kann.

Overflow-Eigenschaften:
`overflow: visible` (default)
`overflow: hidden`
`overflow: auto`
`overflow: scroll` (vertical & horizontal)

# PRAXISPROJEKT

APPLE-PAGE

# Tabellen

Tabellen werden nur zur Darstellung von tabellarischen Inhalten eingesetzt.
NICHT zur Strukturierung der Seite!!!

Grundstruktur:

```
<table border="1">
   <thead>
     <th>Überschrift1</th>
     <th>Überschrift2</th>
   </thead>
   <tbody>
      <tr>
         <td>Zeile 1 - Spalte 1</td>
         <td>Zeile 1 - Spalte 2</td>
      </tr>
      <tr>
        <td>Zeile 2 - Spalte 1</td>
        <td>Zeile 2 - Spalte 2</td>
      </tr>
      <tr>
       <td>Zeile 3 - Spalte 1</td>
       <td>Zeile 3 - Spalte 2</td>
     </tr>
   </tbody>
</table>
```

Aufgabe_18

# Tabellen . Zellen verbinden

Zellen einer Tabelle können sowohl horizontal wie auch vertikal verbunden werden.

Horizontale Verbindung: <td colspan="2">…</td>
Vertikale Verbindung: <td rowspan="2">…</td>

| Mitarbeiter:in | | Adresse | |
|---|---|---|---|
| Müller | Andreas | Schulstrasse 2 | 3100 Hannover |

## Beispiele

Horizontal:

```
<tr><td colspan="2">Mitarbeiter:in</td><td>…</td><td colspan="2">Adresse</td><td>…</td></tr>
<tr><td>Müller</td><td>Andreas</td><td>Schulstrasse 2</td><td>3100 Hannover</td></tr>
```

Vertikal:

```
<tr><td>Spalte 1</td><td>Spalte 2</td><td>Spalte 3</td></tr>
<tr><td>Eintrag 1</td><td>Eintrag 2</td><td rowspan="2">Vertikale verbundene Spalte 3</td></tr>
<tr><td>Eintrag 3</td><td>Einstrag 4</td><td>…</td></tr>
```

| Spalte 1 | Spalte 2 | Spalte 3 |
|---|---|---|
| Eintrag 1 | Eintrag 2 | Vertikal verbundene Spalte 3 |
| Eintrag 3 | Eintrag 4 | |

# Tabellen . Zellen verbinden

<table>
<tr><td>Nicht verbunden</td><td colspan="2">Horizontal verbunden</td><td rowspan="2">Vertikal verbunden</td></tr>
<tr><td>Nicht verbunden</td><td>Nicht verbunden</td><td>Nicht verbunden</td></tr>
<tr><td>Nicht verbunden</td><td colspan="2" rowspan="2">Horizontal & vertikal verbunden</td><td>Nicht verbunden</td></tr>
<tr><td>Nicht verbunden</td><td>Nicht verbunden</td></tr>
</table>

Aufgabe_19

# INTERNAL CSS

NACH INLINE KOMMT INTERNAL

# Bildformate in der Übersicht

....

# Hintergründe festlegen

Elemente können mit Hintergründen versehen werden.

Grundsätzliche Varianten:

```
background-color: red;

background:linear-gradient(to left | right | top | bottom, blue, red);

background:url(./src/images/bild.jpg);
```

Tip: https://css-gradient.io
zum Erstellen eigener Gradienten

Aufgabe_21

# border

Elemente können in HTML Rahmen haben.

Lorem

Lorem

Lorem ipsumg elolita dorum est.

Lorem

Lorm

Aufgabe_21-2

# Inhalte „floaten“

Die Eigenschaft float ermöglicht es zum Beispiel, dass Text um Bilder „fließen“.

Varianten:

```
float: left
float: right
float: none
```

Die Floateinstellungen wirken sich auf die nachfolgenden Elemente aus.
Um dies zu unterbinden, gibt es folgende `clear`-Varianten, die an passender Stelle vergeben werden kann:

```
clear: left
clear: right
clear: both
```

Aufgabe_22

# Alles eine Frage der Position

Von „statischer Positionierung“ spricht man, wenn es keine Positionsangaben gibt.
Beispiel: 4 DIVs nacheinander.
Diese können nur nacheinander angeordnet – wenn es keine weiteren Angaben gibt.

Weitere Positionierungsarten eröffnen weitere Möglichkeiten:

## `position:relative`

Eigenschaften: `top, right, bottom, left` ermöglicht Überlappungen (Offset)

## `position:absolute`

nimmt Elemente aus aktuellen Elementfluss raus, um diese mit `z-index` vor (z-index > 0) oder hinter (z-index < 0) zu positionieren.
Eigenschaften: `top, right, bottom, left, width, height` zur Bestimmung einer definierten Position.

## `position:fixed`

wie absolute Positionierung mit Orientierung immer am aktuellen Viewport. Scrollen nicht mit Content mit. Geeignet für sog. Overlays, die über anderen Elementen liegen. Eigenschaft wie :absolute

## `position:sticky`

Mischung aus relativer und fixer Positionierung. Häufig für Navigationsleisten genutzt, die zunächst an einer weiter unterhalb des oberen Seitenrandes liegenden Stelle positioniert sind, um beim herunterscrollen dann sichtbar zu bleiben.

Aufgabe_23

# SELEKTOREN

WIR SIND DOCH EINE FAMILIE

# Selector in der Übersicht

Definition ist internal oder external möglich. Inline ist es überflüssig.

Typ-Selektoren:

```
body {….}
```

Class-Selektoren:

```
.myclass {…}
```

ID-Selektoren:

```
#id1 {…}
```

Kind-Selektoren:

```
div > p {…}
```

Geschwister-Selektoren:

```
div > p ~ span {…}
```

Nachfahren-Selektoren:

```
div p {…}
```

Aufgabe_24-leer

# Kommentare

## Im HTML-Code und in CSS-Styles

Im HTML-Code:

```
<!– Mein Kommentar -->
```

auch über mehrere Zeilen (Blockkommentare) möglich

```
<!—
Dies ist mein Kommentar
<p>Hier ist ein Absatz auskommentiert</p>
-->
```

Im CSS:

```
/* Mein Kommentar */
```